

Big-O Made Easy

Big-O is a counting game, not a math test. Find what can grow, count how many times the busiest line runs, and keep only the biggest piece.

The one idea

Big-O asks: **if I give this code 10 times more data, how much more work does it do?** That's all. It does not ask how fast your computer is or how many seconds the code takes.

Picture yourself doing the job by hand. You're handed a stack of n index cards:

- **Read the top card.** Same effort whether the stack has 10 cards or 10 million. → $O(1)$
- **Find one card by halving the stack again and again** (it's sorted). A million cards only takes about 20 halvings. → $O(\log n)$
- **Look at every card once.** Twice the cards, twice the work. → $O(n)$
- **Compare every card with every other card.** Twice the cards, four times the work. → $O(n^2)$

The speed ladder

From fastest to slowest. The step counts show why the ladder matters: the bottom rows explode.

Big-O	Name	Doing it by hand	$n = 10$	$n = 100$	$n = 1,000$
$O(1)$	Constant	Read the top card	1	1	1
$O(\log n)$	Logarithmic	Keep halving the stack	~3	~7	~10
$O(n)$	Linear	Look at every card once	10	100	1,000
$O(n \log n)$	Linearithmic	Split into piles, then merge (good sorts)	~33	~660	~10,000
$O(n^2)$	Quadratic	Every card vs. every card	100	10,000	1,000,000
$O(n^3)$	Cubic	Every trio of cards	1,000	1,000,000	1,000,000,000
$O(2^n)$	Exponential	Every possible subset of cards	1,024	31 digits long	302 digits long
$O(n!)$	Factorial	Every possible ordering	3,628,800	158 digits long	2,568 digits long

According to the class worksheet, assessment answers are mostly **$O(1)$** , **$O(n)$** , **$O(n^2)$** , and **$O(n^3)$** , with $O(\log n)$ and $O(n \log n)$ showing up for halving loops and sorts.

The decoder: five steps

Use these five questions on any code, whether it's pseudocode or Python. They work because Big-O is just counting how many times the busiest line runs.

1. **What is n ?** Find the input that can grow: a list's length, or a number passed in. If every loop runs a **fixed** number of times (10, 5, even 100,000,000), nothing grows, so the answer is **$O(1)$** .
2. **Count each loop on its own.** For every `for` or `while`, ask: *how many times does this run as n gets big?* Use the cheat sheet below. Lines that aren't in a loop count as 1, so ignore them.
3. **Loop inside a loop? Multiply.** An n -loop holding an n -loop is $n \times n = O(n^2)$. Three deep is $n \times n \times n = O(n^3)$.
4. **Loops one after another? Add, then keep the biggest.** An n -loop followed by another n -loop is $n + n = 2n$, which is still $O(n)$. An n -loop followed by an n^2 nest is $O(n^2)$.
5. **Clean up.** Drop numbers in front ($5n \rightarrow n$) and drop smaller pieces ($n^2 + n \rightarrow n^2$). If a loop calls a function, replace the call with that function's Big-O and multiply.

Loop-counting cheat sheet

This is the key to step 2. Look at **how the loop counter changes** and **where it stops**.

The loop looks like...	Runs about...	Counts as
Counter goes up by 1 until n	n times	n
Counter goes up by 2 (or 5, or any fixed amount) until n	$n/2$ times	n (constants drop)
Counter goes down by 1 from n to 0	n times	n
Counter doubles ($i = i * 2$) until n	about $\log n$ times	$\log n$
Number is halved ($n = n / 2$) until 1	about $\log n$ times	$\log n$
Runs a fixed number of times (until 10, until 100,000,000)	fixed	1

The loop looks like...	Runs about...	Counts as
Inner loop runs until the outer counter ($j < i$)	0, 1, 2, ... up to $n - 1$ times, which is about $n^2/2$ total	n (and nested, so $n \times n = n^2$)
Loops over a second input m	m times	m (so $n \times m$ when nested)

Two shortcuts that settle most exam questions:

- **Adding or subtracting** a counter $\rightarrow$ linear (n). **Multiplying or dividing** it $\rightarrow$ logarithmic ($\log n$).
- Often you can just **count how deeply the loops are nested**: 0 deep is $O(1)$, 1 deep is $O(n)$, 2 deep is $O(n^2)$, 3 deep is $O(n^3)$. Then check the cheat sheet for halving or fixed loops, which break the pattern.

Worked examples

Each example is decoded with the five steps. Cover the decode, try it yourself, then check.

Example 1: no loop

```
GetFirstAndLast(list)
    first = list[0]
    last = list[length(list) - 1]
    return first + last
```

Decode: n is the list's length. There are no loops, and each line runs once no matter how long the list is. Grabbing an item by index is one step. **Answer: $O(1)$.**

Example 2: a fixed loop

```
PrintTen(list)
    for i from 0 to 9
        Put list[i] to output
```

Decode: The loop always runs exactly 10 times, even if the list has a million items. A fixed count is a constant. **Answer: $O(1)$.**

Example 3: one loop

```
Sum(list)
  total = 0
  for each item in list
    total = total + item
  return total
```

Decode: One loop that visits every item once, so it runs n times. The lines outside the loop run once each and don't matter. **Answer: $O(n)$.**

Example 4: a loop that skips

```
for i from 0 to n - 1, step 3
  Put i to output
```

Decode: The counter goes up by 3, so the loop runs about $n/3$ times. Dividing by 3 is a constant, and constants drop. **Answer: $O(n)$.** Going up by a fixed amount is still linear.

Example 5: two loops, one after the other

```
for each item in list
  Put item to output
for each item in list
  Put item * 2 to output
```

Decode: Each loop runs n times, and they sit side by side, not inside each other. Add them: $n + n = 2n$. Drop the 2. **Answer: $O(n)$.** Side-by-side loops add; they don't multiply.

Example 6: a loop inside a loop

```
for i from 0 to n - 1
  for j from 0 to n - 1
    Put (i, j) to output
```

Decode: The outer loop runs n times. *For each one of those*, the inner loop runs n times. Multiply: $n \times n$. **Answer: $O(n^2)$.** Picture a grid with n rows and n columns; the code visits every cell.

Example 7: the inner loop gets longer each time

```
for i from 0 to n - 1
  for j from 0 to i
    Put j to output
```

Decode: The inner loop runs 1 time, then 2, then 3, ... up to n . That adds up to about $n^2/2$ (half the grid, a triangle). Drop the $1/2$. **Answer: $O(n^2)$.** Don't let "to i " fool you into thinking it's faster.

Example 8: three loops deep

```
for i from 1 to n
  for j from 1 to n
    for k from 1 to n
      count = count + 1
```

Decode: Three nested loops, each running n times: $n \times n \times n$. **Answer: $O(n^3)$.**

Example 9: halving

```
while n > 1
  n = n / 2
```

Decode: Each pass cuts n in half. Starting at 1,000: 500, 250, 125, ..., 1, about 10 passes. Starting at 1,000,000 takes only about 20. The question "how many times can I halve n ?" is exactly what $\log n$ means. **Answer: $O(\log n)$.**

Example 10: doubling

```
i = 1
while i < n
  i = i * 2
```

Decode: i goes 1, 2, 4, 8, 16, ... and reaches n after about $\log n$ doublings. Doubling up to n is the same as halving down from n . **Answer: $O(\log n)$.**

Example 11: a loop that calls a function

```
for i from 0 to n - 1
  BinarySearch(list, target)  // BinarySearch is  $O(\log n)$ 
```

Decode: The loop runs n times, and each time it does an $O(\log n)$ job. A job inside a loop multiplies: $n \times \log n$. **Answer: $O(n \log n)$.**

Example 12: a fixed inner loop

```
for i from 0 to n - 1
  for j from 0 to 4
    Put (i, j) to output
```

Decode: Nested, but the inner loop always runs 5 times. That's $n \times 5 = 5n$, and the 5 drops. **Answer: $O(n)$.** Nesting only gives n^2 when *both* loops grow with n .

Example 13: two different inputs

```
for each a in listA
  for each b in listB
    if a == b
      Put a to output
```

Decode: Two separate lists, of sizes n and m . Nested, so multiply. **Answer: $O(n \times m)$.** If the question says both lists are size n , that's $O(n^2)$.

Example 14: an early exit

```
Find(list, target)
  for each item in list
    if item == target
      return true
  return false
```

Decode: It might find the target on the first try (best case, $O(1)$). But Big-O means the **worst case**: the target is last or missing, so it checks all n items. **Answer: $O(n)$.**

Example 15: the same idea in real Python

The exam may show real Python instead of pseudocode. The syntax changes; the counting doesn't.

```
total = 0
for i in range(n):
    for j in range(n):
        total += i * j
```

Decode: Read `for i in range(n):` as "i from 0 up to $n - 1$ ", which is n times. It holds a second `range(n)` loop, so multiply: $n \times n$. **Answer: $O(n^2)$.** Watch for `range(10)` or another fixed number, which counts as a constant, and for `range(i)`, which depends on the outer loop (Example 7).

The class worksheet, decoded

These are the five problems from the class Big-O worksheet. Each is shown first in readable pseudocode, then decoded. The worksheet key: 1-C, 2-D, 3-D, 4-B, 5-A.

Problem 1

The worksheet writes this loop as `for i=1 and 1<=n`, which is garbled. It means "for i from 1 to n."

```
for i from 1 to n
    j = 1
    while j < n
        j = j + 2
```

Decode: The outer loop runs n times. The inner while adds 2 each pass, so it runs about $n/2$ times, which counts as n (cheat sheet: "goes up by a fixed amount"). The inner loop is nested inside the outer, so multiply: $n \times n$. **Answer: C, $O(n^2)$.**

Problem 2

```
n = 100,000,000
for i from 0 to n - 1
    Put "Hello" to output
```

Decode: This is the trap question. n is **set to a fixed number inside the code**; it isn't an input that can grow. The loop always runs exactly 100,000,000 times. A big number is still a fixed number, so the work never changes. **Answer: D, $O(1)$.** Answer A, $O(100,000,000)$, is a distractor: Big-O never keeps constants, and every constant becomes 1.

Problem 3

```
j = 0
for i from 0 to n - 1
    j = i + j
```

Decode: One loop that runs n times. The math inside ($j = i + j$) is one step, no matter how big the numbers get. Don't be distracted by the loop body doing arithmetic. **Answer: D, $O(n)$.**

Problem 4

```
j = 0
for i from 0 to n - 1
    j = someMethod(n)    // given: someMethod is  $O(\log n)$ 
```

Decode: The loop runs n times. Each pass calls an $O(\log n)$ method. A call inside a loop multiplies: $n \times \log n$. **Answer: B, $O(n \log n)$.**

Problem 5

```
while n > 1
    n = n / 2
```

Decode: n is cut in half each pass, the textbook halving loop from Example 9. **Answer: A, $O(\log n)$.**

Sort and search reference

Memorize this table. It follows the class worksheet's assessment conventions: radix sort is $O(n)$, and bucket sort averages $O(n)$. The exam asks about both average and worst cases, but writes Big-O for both, so read the question for the word "average" or "worst."

Algorithm	Best (min)	Average	Worst (max)	Fast?	How to spot it in code
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	No	Values are swapped with their neighbor
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Finds the minimum and moves it to the front
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	No	Shifts items right to insert into a sorted part

Algorithm	Best (min)	Average	Worst (max)	Fast?	How to spot it in code
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	Yes	Keyword " pivot " or "partition"
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	The list is continually " split " in half, then merged
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	Builds a heap , removes the root repeatedly
Bucket sort	$O(n)$	$O(n)$	$O(n^2)$	Yes	Values are distributed into " buckets "
Radix sort	$O(n)$	$O(n)$	$O(n)$	Yes	Sorts by least/most significant digit
Linear search	$O(1)$	$O(n)$	$O(n)$		Checks items one by one
Binary search	$O(1)$	$O(\log n)$	$O(\log n)$		Checks the middle , discards half

How to remember it: three groups.

- **The slow three** (bubble, selection, insertion): $O(n^2)$. They compare items one pair at a time. Bubble and insertion get lucky to $O(n)$ on already-sorted data; selection never does.
- **The fast three** (merge, heap, quick): $O(n \log n)$. They split the work into halves. Quicksort is the odd one out: a bad pivot pushes its worst case up to $O(n^2)$.
- **The no-compare two** (bucket, radix): $O(n)$. They sort by putting values into slots instead of comparing. Bucket's worst case is $O(n^2)$, when everything lands in one bucket.

Average and worst differ in only two sorts: **quicksort and bucket sort**. If a question asks where they differ, it's one of those.

Why the radix and bucket numbers differ from the main guide: the main guide writes $O(nk)$ and $O(n + k)$, which count digits and buckets. The worksheet says to use $O(n)$ on the assessment, so go with this table for exam answers.

Practice problems

Run the five steps on each one, and write your answer before checking the bottom of this section. Some are written in real Python on purpose, since the exam may show Python instead of pseudocode.

P1.

```

for i from 1 to n
    Put i to output
for i from 1 to n
    for j from 1 to n
        Put i * j to output

```

P2.

```

x = n * n
Put x to output

```

P3.

```

i = n
while i > 0
    i = i - 1

```

P4.

```

i = n
while i > 0
    i = i / 3          // whole-number division

```

P5. (Python)

```

for i in range(n):
    for j in range(i, n):
        for k in range(10):
            count += 1

```

P6. (Python)

```

i = 1
while i < n:
    for j in range(n):
        total += 1
    i = i * 2

```

P7.

```

for i from 0 to 999
  for j from 0 to 999
    Put i + j to output

```

P8. Assume Contains does a linear search through a list of n items.

```

for each item in listA      // listA has n items
  if Contains(listB, item)  // listB has n items
    Put item to output

```

P9.

```

CountDown(n)
  if n <= 0
    return
  Put n to output
  CountDown(n - 1)

```

P10.

```

for i from 1 to n
  for j from 1 to n
    for k from 1 to n
      C[i][j] = C[i][j] + A[i][k] * B[k][j]

```

P11. Which two sorts are $O(n \log n)$ in the best, average, and worst case?

P12. What are quicksort's average and worst cases, and what causes the worst case?

Answers

#	Answer	Why, in one line
P1	$O(n^2)$	n , then $n \times n$, side by side: $n + n^2$, keep the biggest
P2	$O(1)$	No loop; multiplying is one step however big n is
P3	$O(n)$	Counting down by 1 is still n passes
P4	$O(\log n)$	Dividing by 3 is like halving: $\log n$ passes
P5	$O(n^2)$	$n \times (\text{about } n/2) \times 10$; constants drop, leaving n^2
P6	$O(n \log n)$	Outer doubles ($\log n$), inner runs n : $\log n \times n$

#	Answer	Why, in one line
P7	$O(1)$	Both loops are fixed at 1,000; nothing depends on n
P8	$O(n^2)$	n passes, each doing an $O(n)$ search: $n \times n$
P9	$O(n)$	One call per number from n down to 1: n calls, each one step
P10	$O(n^3)$	Three nested n -loops (this is matrix multiplication)
P11	Merge sort and heap sort	Quicksort can hit $O(n^2)$
P12	Average $O(n \log n)$, worst $O(n^2)$	A bad pivot, like always picking the last item of already-sorted data, makes every split lopsided

The worksheet's math rules, in plain words

The worksheet states these as formulas. Each one is the same idea: when n is huge, only the fastest-growing piece matters.

Rule	Plain words	Example
Ignore numbers in front (coefficients)	Doing a job 7 times over is still the same <i>kind</i> of growth	$7n^2 \rightarrow O(n^2)$
Ignore log bases	$\log_2$, $\log_{10}$, $\log_{20}$ all grow the same way, so just write $\log n$	$3 \log_{20} n \rightarrow O(\log n)$
Adding: keep the slowest	Side-by-side work: the biggest piece swamps the rest	$0.01n^3 + 12n^2 - 2 \log n \rightarrow O(n^3)$
Multiplying: multiply the Big-Os	Work inside work (a loop or call inside a loop)	$(n^2 + 2) \times \log n \rightarrow O(n^2 \log n)$

Why " $0.01n^3$ " still wins over " $12n^2$ ": at $n = 10,000$, $0.01n^3$ is 10 billion and $12n^2$ is 1.2 billion. Keep growing n and the gap only widens.

Notation you'll see

- $f(n) = O(g(n))$, $f(n)$ is $O(g(n))$, and $f(n) \in O(g(n))$ all mean the same thing: " f grows no faster than g ." The exam uses the $=$ form.

- **Θ (Theta)** in the worksheet's sort table means the typical (average) growth. The exam writes O for both average and worst, so don't let the symbol trip you up.
- **O is an upper limit.** Technically an $O(n)$ algorithm is also $O(n^2)$ (it's no slower than n^2), but exam answers want the **tightest** fit, the smallest Big- O that's still true.

The worksheet also lists polylogarithmic and other rare classes; its own footnote says you don't need them for C949.